WEEK 5 – Backend & REST Foundations

A comprehensive guide to building professional, scalable REST APIs

MODULE 1

Backend Fundamentals & REST APIs

📘 What is Backend?

The backend is the server-side layer of your application that handles the heavy lifting behind the scenes. It's responsible for business logic, data persistence, authentication, authorization, and serving data to the frontend through APIs.

Core Components:

- **Business Logic:** The rules and workflows that define how your application operates
- **Database Layer:** Persistent storage for user data, application state, and records
- **APIs:** Interfaces that allow frontend applications to communicate with the backend
- **Validation:** Input sanitization and data integrity checks
- **Security:** Authentication, authorization, encryption, and protection against attacks

The frontend communicates with the backend exclusively through **HTTP requests**, making the API design critical for application success.

Example HTTP Requests:

```
GET /users          # Retrieve all users
POST /orders        # Create a new order
PUT /users/10       # Update user with ID 10
DELETE /orders/20   # Delete order with ID 20
```

📘 What is REST?

REST stands for **Representational State Transfer** — an architectural style for designing networked applications. REST isn't a protocol or standard; it's a set of constraints that, when applied correctly, create predictable, scalable, and maintainable APIs.

Core REST Principles:

- **Use Proper HTTP Methods:** GET for reading, POST for creating, PUT/PATCH for updating, DELETE for removing
- **Resource-Based URLs:** URLs represent resources (nouns), not actions (verbs)
- **Stateless:** Each request contains all necessary information; server doesn't store client state
- **JSON Responses:** Standardized data format for easy parsing and consumption
- **Proper Status Codes:** HTTP status codes communicate the outcome of requests

✅ **Correct RESTful Design:**

```
GET /users/10      # Get user with ID 10
POST /users        # Create a new user
```

❌ **Poor Design (Not RESTful):**

```
GET /getUser?id=10 # Action-based, not resource-based
POST /createUser # Verb in URL is redundant
```

HTTP Methods Deep Dive

GET

Purpose: Retrieve data without modifying server state

Characteristics:

- Idempotent (same result every time)
- Cacheable
- Safe (no side effects)
- Parameters in URL query string

Example: `GET /users/42`

POST

Purpose: Create new resources

Characteristics:

- Not idempotent (creates new resource each time)
- Data sent in request body
- Returns 201 Created on success
- Should return created resource

Example: `POST /users`

PUT

Purpose: Full replacement of existing resource

Characteristics:

- Idempotent
- Requires complete resource data
- Creates if doesn't exist
- Overwrites all fields

Example: `PUT /users/42`

PATCH

Purpose: Partial update of existing resource

Characteristics:

- Idempotent
- Only modified fields required
- More flexible than PUT
- Preferred for updates

Example: `PATCH /users/42`

DELETE

Purpose: Remove resources

Characteristics:

- Idempotent
- May return 204 No Content
- Subsequent calls return 404
- Should be irreversible

Example: `DELETE /users/42`

Pro Tip: Choose PUT when you're replacing the entire resource, and PATCH when you're only updating specific fields. For example, updating just an email address should use PATCH, while replacing all user details should use PUT.

HTTP Status Codes Explained

HTTP status codes are three-digit numbers that communicate the result of an HTTP request. Understanding and using them correctly is essential for building predictable, debuggable APIs. They're grouped into five categories, each with a specific meaning.

2xx Success

200 OK: Request succeeded, response contains requested data

201 Created: Resource successfully created (use with POST)

204 No Content: Success but no response body (common with DELETE)

When to use: Everything worked as expected

4xx Client Errors

400 Bad Request: Invalid request format or missing required fields

401 Unauthorized: Authentication required or failed

403 Forbidden: Authenticated but lacks permission

404 Not Found: Requested resource doesn't exist

422 Unprocessable Entity: Valid format but semantic errors

When to use: Client made a mistake in their request

5xx Server Errors

500 Internal Server Error: Unexpected server-side failure

502 Bad Gateway: Invalid response from upstream server

503 Service Unavailable: Server temporarily down

504 Gateway Timeout: Upstream server didn't respond in time

When to use: Server encountered an unexpected problem

Common Mistakes to Avoid

- ~~Returning 200 for errors~~ — Always use appropriate error codes
- ~~Using 500 for validation errors~~ — That's a 400 Bad Request
- ~~Using 404 for authorization issues~~ — Use 401 or 403 instead
- ~~Inconsistent status codes~~ — Same error should return same code

Best Practices

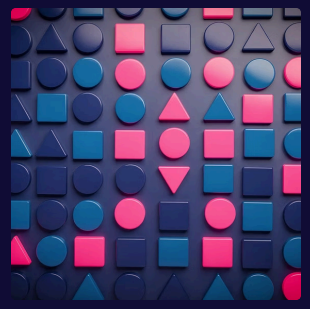
- Return **201 Created** with the new resource's location in the Location header
- Use **400** for input validation failures with descriptive error messages
- Reserve **500** for genuine server failures, not business logic errors
- Log all 5xx errors for debugging and monitoring

Why API Design Matters

Poor API design creates technical debt that compounds over time. What seems like a small inconsistency today becomes a maintenance nightmare tomorrow. Professional architects understand that API design is an investment in your application's future scalability, maintainability, and developer experience.

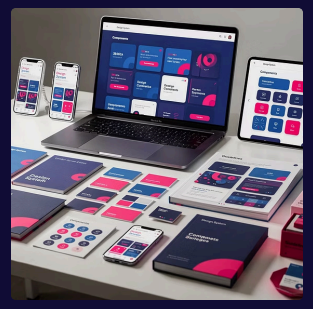
Naming Clarity

Use descriptive, self-documenting resource names that immediately convey their purpose. `/users` is clear; `/data` is not. Plural nouns for collections (`/orders`), singular for specific items (`/orders/123`). Avoid abbreviations unless universally understood.



Predictability

Developers should be able to guess your API's behavior without reading documentation. If `GET /users` returns a list, `GET /products` should too. If pagination uses `?page=1&limit=20` for users, use the same pattern everywhere. Consistency eliminates surprises.



Consistency

Establish conventions and follow them religiously. Use the same naming patterns, response structures, error formats, and authentication methods across all endpoints. Consistency reduces cognitive load and makes your API feel professional and polished.

Scalability

Design for future growth from day one. Plan for versioning (`/v1/users`), pagination, filtering, and sorting. Consider how your API will handle millions of users, not just hundreds. Avoid design choices that lock you into limitations you'll regret later.

> "Bad API design is like building a house on a weak foundation. You might not notice the cracks at first, but eventually, the entire structure becomes unstable. Invest in solid design principles early, and your future self will thank you."

Real-World Impact: A well-designed API reduces onboarding time for new developers, minimizes bugs, improves frontend productivity, and makes your application easier to maintain and extend. Conversely, poorly designed APIs lead to confusion, workarounds, technical debt, and frustrated teams.

EXERCISE

Exercise: Design APIs for User Module

In this hands-on exercise, you'll design a complete set of RESTful API endpoints for a user management system. This is your opportunity to apply REST principles, HTTP methods, and status codes to a real-world scenario. Focus on creating clear, predictable, and scalable endpoint designs.

📝 Create User

Design an endpoint to register a new user account

Consider: Required fields, password hashing, email validation, duplicate prevention

🔍 Get User

Design an endpoint to retrieve a specific user's details

Consider: User ID parameter, sensitive data filtering, authorization checks

✏️ Update User

Design an endpoint to modify user information

Consider: Partial vs. full updates, field validation, permission checks

🗑️ Delete User

Design an endpoint to remove a user account

Consider: Soft vs. hard delete, cascading effects, authorization requirements

📋 List Users

Design an endpoint to retrieve multiple users with pagination

Consider: Page size limits, sorting options, filtering capabilities, total count

Requirements

- Use appropriate HTTP methods for each operation
- Follow RESTful URL naming conventions
- Implement pagination for list endpoint
- Include query parameters for filtering and sorting
- Define clear request/response formats
- Specify all possible status codes
- Handle edge cases (not found, validation errors, duplicates)

Activity Details

Type: Individual Design Exercise

Duration: 45-60 minutes

Deliverable: API Design Document (see next card)

Tools: Text editor, API design tools (optional)

Collaboration: Discuss design choices with peers, but submit individual work

Expected Output: API Design Document

Your API design document should be comprehensive, clear, and follow industry standards. Each endpoint must be fully specified with all necessary details. Use the template below as a guide for structuring your submission.

01

Endpoint URL

Full resource path including any path parameters

02

HTTP Method

GET, POST, PUT, PATCH, or DELETE

03

Request Format

Body structure, headers, query parameters

04

Response Format

Success response structure with example data

05

Status Codes

All possible outcomes with explanations

Example Entry Format

Endpoint: POST /api/v1/users

Purpose: Create a new user account

Request Body:

```
{
  "email": "user@example.com",
  "password": "SecurePass123!",
  "firstName": "Jane",
  "lastName": "Doe",
  "role": "user"
}
```

Success Response (201 Created):

```
{
  "success": true,
  "data": {
    "id": 123,
    "email": "user@example.com",
    "firstName": "Jane",
    "lastName": "Doe",
    "role": "user",
    "createdAt": "2024-01-15T10:30:00Z"
  }
}
```

Status Codes:

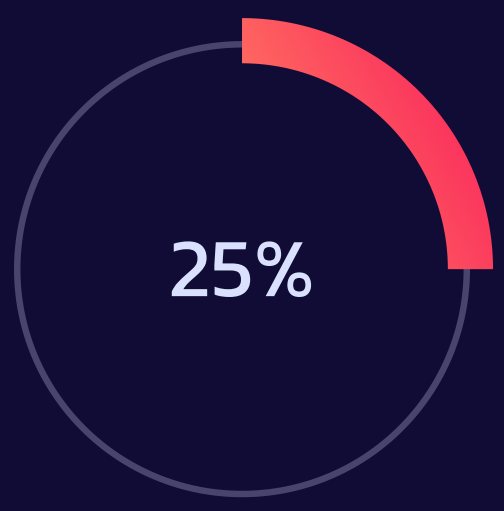
- **201 Created:** User successfully created
- **400 Bad Request:** Invalid email format or missing required fields
- **409 Conflict:** Email already registered
- **500 Internal Server Error:** Database connection failure

Additional Requirements: Include pagination parameters for list endpoint (?page=1&limit=20), document all query parameters for filtering and sorting, specify authentication requirements, and include notes on any business logic or validation rules.

ASSESSMENT

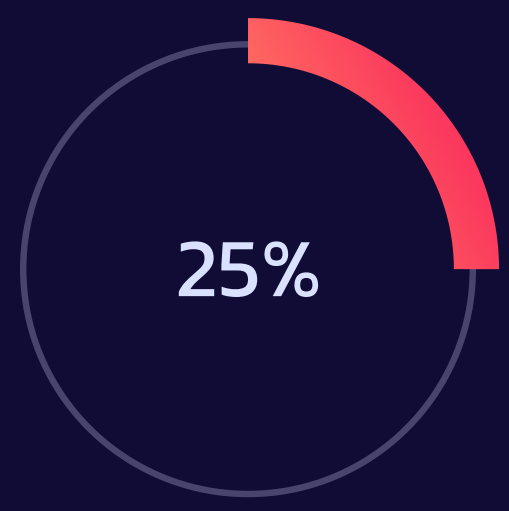
Evaluation Criteria

Your API design will be evaluated across four key dimensions. Each criterion reflects industry best practices and real-world requirements. Aim for excellence in all areas to demonstrate mastery of REST API design principles.



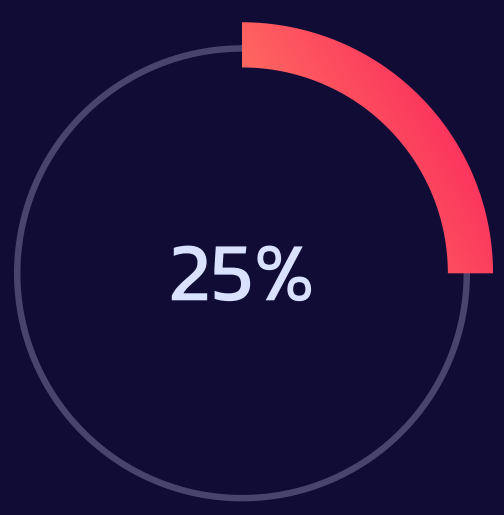
Correct HTTP Usage

Appropriate methods for each operation



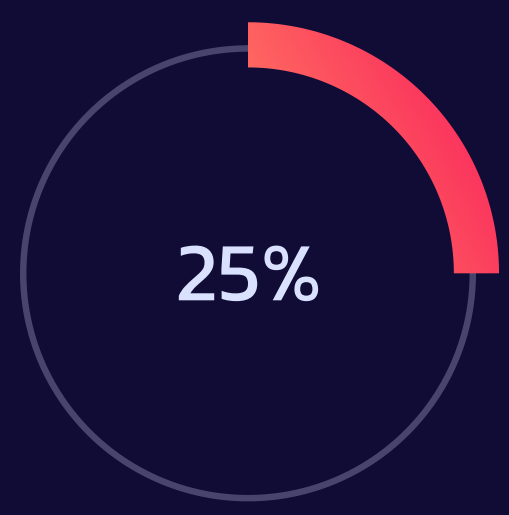
Proper Naming

Clear, consistent, RESTful conventions



RESTful Structure

Resource-based URLs, stateless design



Clear Responses

Well-formatted, predictable output

Assessment Type

Format: Written Design Review

Weight: Formative Assessment (Practice)

Feedback: Detailed written feedback on each endpoint

Iteration: Opportunity to revise based on feedback

Detailed Rubric

Excellent (90-100%):

- All endpoints follow REST principles flawlessly
- Comprehensive status code coverage
- Clear, consistent naming throughout
- Pagination and filtering properly implemented
- Edge cases thoroughly considered

Good (75-89%):

- Most endpoints follow REST principles
- Common status codes included
- Minor naming inconsistencies
- Pagination present but incomplete

Needs Improvement (<75%):

- Incorrect HTTP method usage
- Non-RESTful URL structure
- Missing status codes or response formats
- No pagination or filtering

MODULE 2

REST API Implementation

Now that you understand REST principles, it's time to bring your designs to life with code. You'll build a production-ready API using modern frameworks and best practices. The focus is on clean architecture, separation of concerns, and maintainable code that scales.

Framework Options

Node.js + Express (Beginner-Friendly)

- Lightweight and flexible
- Large ecosystem of middleware
- Minimal boilerplate
- Great for learning fundamentals
- Manual structure setup required

NestJS (Recommended for Production)

- Enterprise-grade architecture
- Built-in dependency injection
- TypeScript-first
- Opinionated structure promotes best practices
- Excellent for scalable applications

Project Structure

```
src/
├── controllers/     # Route handlers
│   └── user.controller.ts
├── services/        # Business logic
│   └── user.service.ts
├── routes/          # API routes
│   └── user.routes.ts
├── models/          # Data models
│   └── user.model.ts
├── middleware/      # Custom middleware
│   ├── auth.middleware.ts
│   └── validation.middleware.ts
├── utils/           # Helper functions
│   └── error-handler.ts
└── app.ts           # Application entry
```

This structure enforces separation of concerns and makes your codebase easy to navigate and maintain.

Key Implementation Rules

No Business Logic in Controllers

Controllers should only handle HTTP concerns (parsing requests, sending responses). All business logic belongs in services. This keeps controllers thin and makes testing easier.

Use Service Layer

Services contain your application's business logic, interact with databases, and implement your core functionality. They're framework-agnostic and reusable across different contexts.

Validate Input

Never trust user input. Use validation libraries like Joi, class-validator, or Zod to ensure data meets your requirements before processing. Fail fast with clear error messages.

Handle Errors Centrally

Implement global error handling middleware to catch all exceptions, format error responses consistently, log failures, and prevent sensitive information leakage in production.

ASSIGNMENT

Implementation Assignment

Transform your API design from the previous exercise into a fully functional REST API. This assignment challenges you to apply everything you've learned about backend development, from routing to error handling to proper HTTP semantics.

Setup

Initialize project, install dependencies, configure database connection

Implement CRUD

Build Create, Read, Update, Delete operations for user resource

Add Features

Integrate status codes, validation, pagination, error handling

Test & Deploy

Write tests, verify all endpoints, prepare documentation

Core Requirements

1. CRUD Operations

- `POST /users` — Create user with validation
- `GET /users/:id` — Get user by ID
- `GET /users` — List users with pagination
- `PATCH /users/:id` — Update user fields
- `DELETE /users/:id` — Delete user

2. Proper Status Codes

- 201 Created for successful POST
- 200 OK for successful GET/PATCH
- 204 No Content for successful DELETE
- 400 Bad Request for validation errors
- 404 Not Found for missing resources
- 500 Internal Server Error for crashes

3. Pagination Implementation

- Support `?page=1&limit=20` query params
- Return total count in response
- Default to reasonable limits (e.g., 10 items)
- Maximum limit enforcement

Technical Specifications

Expected Deliverable:

- Working API codebase pushed to GitHub
- README with setup instructions
- Postman collection or API documentation
- Environment configuration files

Assessment Type: Practical Coding Assignment

Weight: Summative (Major Assessment)

Submission Format: GitHub repository link

Evaluation Criteria:

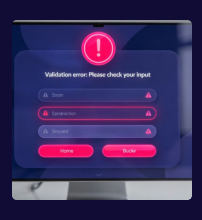
- Code organization and structure (25%)
- Correct implementation of CRUD operations (30%)
- Proper HTTP status codes usage (15%)
- Pagination functionality (10%)
- Error handling and validation (15%)
- Code quality and documentation (5%)

Bonus Points: Authentication middleware, filtering/sorting, comprehensive tests, deployment to cloud platform

MODULE 3

API Failure Handling

Errors are inevitable in any application. What separates professional APIs from amateur ones is how gracefully they handle failures. Your error handling strategy directly impacts developer experience, debugging efficiency, and application reliability.



Validation Errors (400)

User submitted invalid data — missing required fields, incorrect format, out-of-range values, type mismatches. Return specific field-level errors so developers know exactly what to fix.

Example: Email format invalid, password too short, age must be positive integer

Database Errors (500)

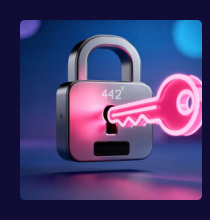
Connection failures, query timeouts, constraint violations, disk space issues. These are server-side problems. Log detailed information internally but return generic messages to clients to avoid exposing internals.

Example: "Service temporarily unavailable" instead of "MySQL connection pool exhausted"

Resource Not Found (404)

The requested resource doesn't exist in the database. Common with ID-based lookups. Return clear messages indicating what wasn't found and potentially suggest alternatives.

Example: "User with ID 999 not found" is better than just "Not found"



Unauthorized Access (401/403)

401: Authentication failed or missing. 403: Authenticated but lacks permissions. Critical distinction — 401 means "log in," 403 means "you can't do that." Never expose why authentication failed for security.

Example: Generic "Invalid credentials" instead of "Password incorrect"

Unexpected Crashes (500)

Unhandled exceptions, null pointer errors, out of memory, undefined variables. These should never happen in production but must be caught gracefully. Always log full stack traces for debugging.

Example: Return safe error message while logging complete error details server-side

Standard Error Response Format

Consistency in error responses is crucial. Establish a standard format and use it everywhere:

```
{
  "success": false,
  "message": "User not found",
  "error_code": "USER_404",
  "timestamp": "2024-01-15T10:30:00Z",
  "path": "/api/v1/users/999"
}
```

For validation errors, include field-specific details:

```
{
  "success": false,
  "message": "Validation failed",
  "error_code": "VALIDATION_ERROR",
  "errors": [
    {
      "field": "email",
      "message": "Invalid email format"
    },
    {
      "field": "password",
      "message": "Must be at least 8 characters"
    }
  ]
}
```

Error Handling Best Practices

- **Use error codes:** Machine-readable codes enable programmatic error handling
- **Human-readable messages:** Clear, actionable messages help developers debug
- **Include timestamps:** Critical for correlating errors with logs
- **Add request path:** Shows exactly which endpoint failed
- **Never expose stack traces:** Security risk in production environments
- **Log everything:** Full details server-side, safe messages client-side
- **Use error monitoring:** Tools like Sentry catch errors you didn't anticipate

Security Note: Never include sensitive information in error messages. Database connection strings, API keys, file paths, and internal implementation details should only appear in server logs, never in API responses.

Architect-Level Error Thinking

Senior architects don't just handle errors — they design systems where failures are predictable, consistent, and debuggable. This mindset shift transforms error handling from an afterthought into a core architectural principle that shapes your entire API design.

Predictability

Every error scenario should have a defined, documented response. Developers using your API shouldn't encounter surprises. If endpoint X returns 400 for invalid input, endpoint Y should too — same format, same structure, same behavior. Predictable errors reduce integration time and debugging frustration.

Consistency

Establish error response standards and enforce them across your entire API. Same error codes, same response structure, same message format. Use middleware or interceptors to centralize error formatting. Consistency makes your API feel professional and thoughtfully designed, not haphazardly thrown together.

Logging

Every failure must be logged with sufficient context for debugging. Include request ID, user ID, timestamp, endpoint, input parameters, and full error details. Use structured logging (JSON) for easy parsing. Implement log levels (ERROR, WARN, INFO) and monitoring alerts for critical failures. Logs are your time machine when investigating production issues.

Activity: Error Scenario Analysis

Exercise Type: Group Discussion & Design

Duration: 30 minutes

Task: Analyze the following scenarios and design appropriate error responses:

1. User tries to create account with email that already exists
2. API receives malformed JSON in request body
3. Database connection times out during query
4. User attempts to delete another user's resource
5. API rate limit exceeded (too many requests)
6. Required authentication token is missing
7. Pagination offset exceeds total records

For each scenario, specify: HTTP status code, error code, error message, any additional context

Expected Output

Deliverable: Error Handling Documentation

Create a comprehensive error reference guide that includes:

- Complete list of possible error codes
- Status code mapping for each scenario
- Example response JSON for each error type
- When to use each status code
- Client-side handling recommendations

Assessment Criteria:

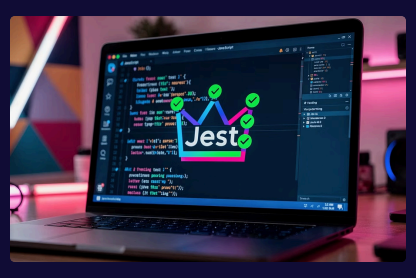
- **Correctness (40%):** Appropriate status codes and error codes
- **Clarity (30%):** Clear, actionable error messages
- **Consistency (20%):** Uniform format across all errors
- **Completeness (10%):** Edge cases considered

Assessment Type: Written Documentation Review

MODULE 4

API Testing Basics

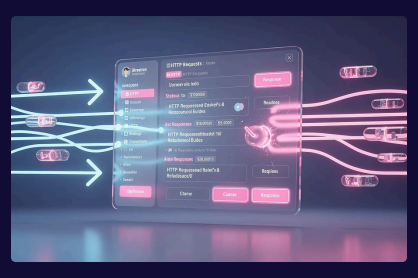
Untested code is broken code — you just don't know it yet. Professional developers write tests to catch bugs before they reach production, document expected behavior, and enable confident refactoring. Let's build a comprehensive testing strategy for your REST APIs.



Jest — Testing Framework

Jest is a JavaScript testing framework that makes writing tests delightful. It provides test runners, assertion libraries, mocking capabilities, and code coverage reports. Fast, parallel test execution. Zero configuration for most projects. Excellent error messages and snapshot testing.

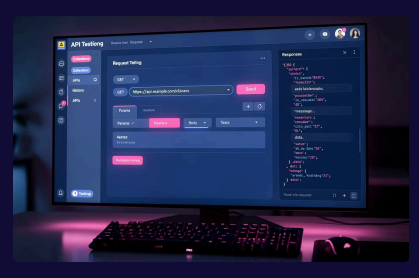
Use for: Unit tests, integration tests, and test suites



Supertest — HTTP Assertions

Supertest is a library for testing Node.js HTTP servers. It allows you to make HTTP requests in tests and assert on responses. No need to start server manually. Chainable assertions. Works perfectly with Jest. Ideal for testing your Express/NestJS APIs.

Use for: API endpoint testing, integration testing



Postman — Manual Testing

Postman is a GUI tool for manual API testing and exploration. Build requests, save collections, automate tests, generate documentation. Great for exploratory testing. Can write JavaScript tests. Team collaboration features. Not ideal for CI/CD but excellent for development.

Use for: Manual testing, API documentation, sharing examples

What to Test: Comprehensive Coverage

✅ Success Case (Happy Path)

Test that everything works when inputs are valid and conditions are perfect. Create user with valid data. Get existing resource. Update with proper permissions. Delete when authorized.

Verify: Correct status code (200/201), expected response structure, data persisted correctly

❌ Invalid Input

Test behavior with malformed, missing, or invalid data. Missing required fields. Wrong data types. Out-of-range values. Malformed JSON. SQL injection attempts.

Verify: 400 Bad Request, clear validation error messages, no data persistence

🔍 Not Found

Test requests for non-existent resources. Get user that doesn't exist. Update missing record. Delete already-deleted resource.

Verify: 404 Not Found, helpful error message, no crashes

🔒 Unauthorized Access

Test authentication and authorization. Missing auth tokens. Invalid credentials. Expired sessions. Insufficient permissions. Access to other users' resources.

Verify: 401/403 status codes, no sensitive data leakage, security maintained

Final Testing Assignment

Task: Write comprehensive tests for your User API

Requirements:

- Minimum 20 test cases covering all endpoints
- Use Jest + Supertest for automated tests
- Include success and failure scenarios
- Test validation, authorization, edge cases
- Achieve >80% code coverage
- Tests must be runnable with `npm test`

Expected Output: Test suite file with passing tests

Assessment Type: Automated Code Testing

Assessment Criteria

30%

Coverage

Breadth of test scenarios

25%

Correctness

Tests verify right behavior

25%

Quality

Well-organized, readable tests

20%

Edge Cases

Handles unusual scenarios

Bonus Points: CI/CD pipeline integration, load testing, mutation testing, API documentation generation from tests

> "Testing is not about finding bugs in code — it's about building confidence that your system behaves correctly under all conditions. Every test is a specification of expected behavior that protects you from future regressions."